



TSwap Protocol Audit Report

Version 1.0

Cyfrin.io

September 9, 2026

TSwap Protocol Audit Report

github.com/nuelonye

September 09, 2026

Prepared by: NuelOnye Lead Auditors: - xxxxxxxx

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Roles
 - Issues found
- Findings
 - High
 - * [H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees
 - * [H-2] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens
 - * [H-3] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing users to receive the incorrect amount of tokens
 - * [H-4] In `TSwapPool::_swap` the extra tokens given to users after every `swapCount` breaks the protocol invariant of $x * y = k$
 - Medium

- * [M-1] `TSwapPool::deposit` is missing deadline check causing transaction to complete even after the deadline
- Low
 - * [L-1] `TSwapPool::_addLiquidityMintAndTransfer` emits event with wrong order of Parameters
 - * [L-2] Default value returned by `TSwapPool::swapExactInput` result in an incorrect return value given
- Informationals
 - * [I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is not used and should be removed in
 - * [I-2] No zero address check in `PoolFactory` and `TSwapPool` constructor
 - * [I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name()`
 - * [I-4] Event is missing `indexed` fields
 - * [I-5] `MINIMUM_WETH_LIQUIDITY` is a constant and not required to be emitted
 - * [I-6] Unused variable in `TSwapPool::deposit` not gas Efficient
 - * [I-7] `TSwapPool::deposit` doesn't follow CEI
 - * [I-8] No natspec in `TSwapPool::swapExactInput`
 - * [I-9] Functions not used within the contract should be external

Protocol Summary

This project is meant to be a permissionless way for users to swap assets between each other at a fair price. You can think of T-Swap as a decentralized asset/token exchange (DEX). T-Swap is known as an Automated Market Maker (AMM) because it doesn't use a normal "order book" style exchange, instead it uses "Pools" of an asset. It is similar to Uniswap. To understand Uniswap, please watch this video: [Uniswap Explained](#)

Disclaimer

The NuelOnye team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: e643a8d4c2c802490976b538dd009b351b1c8dda
- In Scope:

```
1 ./src/  
2 #-- PoolFactory.sol  
3 #-- TSwapPool.sol
```

Roles

- Liquidity Providers: Users who have liquidity deposited into the pools. Their shares are represented by the LP ERC20 tokens. They gain a 0.3% fee every time a swap is made.
- Users: Users who want to swap tokens.

Issues found

Severity	Number of issues found
High	4
Medium	1
Low	2

Severity	Number of issues found
Info	9
Gas	0
Total	16

Findings

High

[H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees

Description: The `getInputAmountBasedOnOutput` function is intended to calculate the amount of tokens a user should deposit given an output token amount. However, the function currently miscalculates the resulting amount. When calculating the fee, it scales the amount by 10_000 instead of 1_000.

Impact: Protocol takes more fees than expected from users.

Proof of Concept:

Proof of Code

Add the following to the `TSwapPool.t.sol` test file

```
1     function test_getInputAmountBasedOnOutput_overcharges() public view
2     {
3         uint256 inputReserves = 1000e18;
4         uint256 outputReserves = 1000e18;
5         uint256 outputAmount = 100e18; // user wants 100 tokens out
6
7         // Wrong calculation function
8         uint256 buggyInputAmount = pool.getInputAmountBasedOnOutput(
9             outputAmount, inputReserves, outputReserves);
10        // Correct Calculation or logic
11        uint256 correctInputAmount = ((inputReserves * outputAmount) *
12            1_000) / ((outputReserves - outputAmount) * 997);
13
14        // Proof
15        assert(buggyInputAmount > (correctInputAmount * 10)); //
16            roughly 10x more
```

```
14     console.log("correctInputAmount: ", correctInputAmount);
15     console.log("buggyInputAmount: ", buggyInputAmount);
16     console.log("Overcharge factor      :", buggyInputAmount /
17               correctInputAmount);
17 }
```

Recommended Mitigation:

```
1     function getInputAmountBasedOnOutput(
2         uint256 outputAmount,
3         uint256 inputReserves,
4         uint256 outputReserves
5     )
6     public
7     pure
8     revertIfZero(outputAmount)
9     revertIfZero(outputReserves)
10    returns (uint256 inputAmount)
11    {
12 -        return ((inputReserves * outputAmount) * 10_000) / ((
13 +        return ((inputReserves * outputAmount) * 1_000) / ((
14            outputReserves - outputAmount) * 997);
15            outputReserves - outputAmount) * 997);
16    }
```

[H-2] Lack of slippage protection in TSwapPool : : swapExactOutput causes users to potentially receive way fewer tokens

Description: The `swapExactOut` function does not include any sort of slippage protection. This function is similar to what is done in `TSwapPool : : swapExactInput` where the function specifies a `minOutputAmount` they are willing to receive for their `inputAmount`, the `swapExactOutput` function should also specify a `maxInputAmount` the user is willing to give for their `outputAmount`.

Impact: If market conditions change before the transaction processes, the user could get a much worse swap.

Proof of Concept: 1. The price of WETH right now is 1,000 USDC 2. User inputs a `swapExactOutput` 1. inputToken = USDC 2. outputToken = WETH 3. outputAmount = 1 4. deadline = ... 3. The function does not offer a maxInput amount 4. As the transaction is pending in the memepool, the market changes! and the price moves HUGE -> 1 WETH is now 10,000 USDC. 10x more than the user expected. 5. Finally, the transaction completes, but the user sent the protocol 10,000 USDC for 1 WETH instead of 1,000 USDC.

Proof Of Code

Code

Add the following to the `TSwapPool.t.sol` test file

```
1      function test_swapExactOutput_noSlippageProtection() public {
2          address attacker = makeAddr("attacker");
3
4          uint256 WETH_AMOUNT = 1_000e18;
5          uint256 POOL_TOKEN_AMOUNT = 1_000_000e18;
6
7          // Give liquidity provider enough to add liquidity
8          weth.mint(LiquidityProvider, WETH_AMOUNT);
9          poolToken.mint(LiquidityProvider, POOL_TOKEN_AMOUNT);
10
11         // 1. addLiquidity of 1_000e18 WETH and 1_000_000e18 USDC
12         // This means currently, 1 WETH (outputToken) is worth 1000
13         // USDC(poolToken / inputToken)
14         vm.startPrank(LiquidityProvider);
15         weth.approve(address(pool), WETH_AMOUNT);
16         poolToken.approve(address(pool), POOL_TOKEN_AMOUNT);
17         pool.deposit(WETH_AMOUNT, WETH_AMOUNT, POOL_TOKEN_AMOUNT,
18             uint64(block.timestamp));
19         vm.stopPrank();
20
21         // 2. Record how much USDC the victim "expects" to pay for WETH
22         // right now
23         // We cant use `getInputAmountBasedOnOutput` because it is
24         // bugged
25         uint256 victimDesiredOutputAmount = 10e18; // User wants
26         // exactly 10 output tokens (Weth)
27         uint256 inputReservesBefore = poolToken.balanceOf(address(pool));
28         uint256 outputReservesBefore = weth.balanceOf(address(pool));
29         uint256 victimExpectedInputAmount = ((inputReservesBefore *
30             victimDesiredOutputAmount) * 1_000)
31             / ((outputReservesBefore - victimDesiredOutputAmount) *
32             997);
33         console.log("Victim expects to pay: ",
34             victimExpectedInputAmount); // at least 10_131
35             .404313951956880743 USDC
36
37         // 3. Attacker front-runs, moves the price drastically
38         uint256 attackerInputAmount = 200_000e18;
39         uint256 minOutputAmount =
40             pool.getOutputAmountBasedOnInput(attackerInputAmount,
41                 inputReservesBefore, outputReservesBefore);
42         vm.startPrank(attacker);
43         poolToken.mint(attacker, attackerInputAmount);
44         poolToken.approve(address(pool), attackerInputAmount);
45         pool.swapExactInput(poolToken, attackerInputAmount, weth,
46             minOutputAmount, uint64(block.timestamp));
```

```
36     console.log("Attackers current WETH balance: ", weth.balanceOf(
37         attacker)); // 166.249791562447890611 WETH
38     vm.stopPrank();
39     // 4. Record how much USDC the victim will actually pay for
40     // WETH after price movement
41     // We cant use `getInputAmountBasedOnOutput` because it is
42     // bugged
43     uint256 inputReserversAfter = poolToken.balanceOf(address(pool)
44         );
45     uint256 outputReservesAfter = weth.balanceOf(address(pool));
46     uint256 victimActualPaidInputAmount = ((inputReserversAfter *
47         victimDesiredOutputAmount) * 1_000)
48         / ((outputReservesAfter - victimDesiredOutputAmount) * 997)
49     ;
50     console.log("Victim actually paid: ",
51         victimActualPaidInputAmount); // at least 14_611
52         .356940114660006422 USDC
53
54     assertGt(victimActualPaidInputAmount, victimExpectedInputAmount
55         );
56 }
```

NOTE: This POC don't actually call the `TSwapPool.swapExactOutput` function because it uses the `getInputAmountBasedOnOutput` function which has a known bug. Hence it only simulates the prices — how much `inputToken` the user should have paid for an `outputToken` versus how much they actually paid. Since the function don't allow for the user to set a limit (`maxInputAmount`), they take whatever price is set.

Recommended Mitigation: We should include a `maxInputAmount` so the user only has to spend up to a specific amount, and can predict how much they will spend on the protocol.

```
1     function swapExactOutput(
2         IERC20 inputToken,
3         IERC20 outputToken,
4         uint256 outputAmount,
5     +     uint256 maxInputAmount
6         uint64 deadline
7     .
8     .
9     .
10    {
11        uint256 inputReserves = inputToken.balanceOf(address(this));
12        uint256 outputReserves = outputToken.balanceOf(address(this));
13    +     if(inputAmount > maxInputAmount) {
14    +         revert();
15    }
16
17        inputAmount = getInputAmountBasedOnOutput(outputAmount,
```

```
        inputReserves, outputReserves);
18
19     _swap(inputToken, inputAmount, outputToken, outputAmount);
20 }
```

[H-3] TSwapPool::sellPoolTokens mismatches input and output tokens causing users to receive the incorrect amount of tokens

Description: The `sellPoolTokens` function is intended to allow users to easily sell pool tokens for WETH. Users indicate how many pool tokens they willing to sell in the `poolTokenAmount` parameter. However, the function currently miscalculates the swapped amount.

This is due to the fact the the `swapExactOutput` function is called whereas the `swapExactInut` function should be called. Because users specify the exact amount of input tokens, not output.

Impact: Users will swap the wrong amount of tokens which is a severe disruption of protocol functionality.

Proof of Concept:

Recommended Mitigation: Consider chainging the implementation to use, `swapExactInput` instead of `swapExactOutput`. Note that this will also require changing the `sellPoolTokens` function to accept a new parameter (ie., `minWethToReceive`) to be passed to `swapExactInput`.

```
1     function sellPoolTokens(
2         uint256 poolTokenAmount
3 +         uint256 minWethToReceive
4     ) external returns (uint256 wethAmount) {
5 -         return swapExactOutput(i_poolToken, i_wethToken,
6 +         return swapExactInput(i_poolToken, poolTokenAmount, i_wethToken
7         , minWethToReceive, uint64(block.timestamp));
8     }
```

Additionally, it might be wise to add a deadline to the function, as there is none currently.

[H-4] In TswapPool::_swap the extra tokens given to users after every swapCount breaks the protocol invariant of $x * y = k$

Description: The protocol follows a strict invariant of $x * y = k$. Where: - x : The balance of the pool token - y : The balance of WETH - k : The constant product of the two balance

This means, that whenever the balances change in the protocol, the ratio between the two amounts should remain contant, hence the k . However, this is broken due to the extra incentive in the `_swwap` function. Meaning that over time the protocol funcnds will be drained.

The following block of code is responsible for the issue.

```
1      swap_count++;
2      if (swap_count >= SWAP_COUNT_MAX) {
3          swap_count = 0;
4          outputToken.safeTransfer(msg.sender, 1
5                                   _000_000_000_000_000_000);
6      }
```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol.

Most simply put, the protocol's core invariant is broken.

Proof of Concept: 1. A user swaps 10 times, and collects the extra incentive of 1_000_000_000_000_000_000 tokens
2. That user continues to swap until all the protocol funds are drained.

Proof Of Code

place the following into `TSwapPool.t.sol` test file

NOTE: The assertion in this POC fails which denotes that the Protocol's core invariant Broke!!

```
1      function testInvariantBroken() public {
2          vm.startPrank(liquidityProvider);
3          weth.approve(address(pool), 100e18);
4          poolToken.approve(address(pool), 100e18);
5          pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
6          vm.stopPrank();
7
8          uint256 outputWeth = 1e17;
9
10         vm.startPrank(user);
11         poolToken.mint(user, 100e18);
12         poolToken.approve(address(pool), type(uint256).max);
13         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
14         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
15         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
16         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
17         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
18         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
19         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
20         pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
```

```
21     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
22         timestamp));  
23     int256 startingY = int256(weth.balanceOf(address(pool)));  
24     int256 expectedDeltaY = int256(-1) * int256(outputWeth);  
25  
26     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.  
27         timestamp));  
28     vm.stopPrank();  
29     uint256 endingY = weth.balanceOf(address(pool));  
30     int256 actualDeltaY = int256(endingY) - int256(startingY);  
31     assertEq(actualDeltaY, expectedDeltaY);  
32 }
```

Recommended Mitigation: Remove the extra incentive mechanisms. If you want to keep this in, we should account for the change in the $x * y = k$ protocol invariant. Or we should set aside tokens in the same way we with fees.

```
1 - swap_count++;  
2 -     if (swap_count >= SWAP_COUNT_MAX) {  
3 -         swap_count = 0;  
4 -         outputToken.safeTransfer(msg.sender, 1  
5 -             _000_000_000_000_000_000);  
6 -     }
```

Medium

[M-1] TSwapPool::deposit is missing deadline check causing transaction to complete even after the deadline

Description: The `deposit` function accepts a `deadline` parameter, which according to documentation, transaction must have been completed before the deadline. However, this parameter is never used. As a consequence, operation that add liquidity to the pool might be executed at unexpected times, in market conditions where the deposit rate is unfavourable.

Impact: Transactions could be sent when market conditions are unfavourable to deposit, even when adding a deadline parameter

Proof of Concept: The `deadline` parameter is unused

Recommended Mitigation: Consider making the following change to the function.

```
1     function deposit(  
2         uint256 wethToDeposit,  
3         uint256 minimumLiquidityTokensToMint,
```

```
4         uint256 maximumPoolTokensToDeposit,
5         uint64 deadline
6     )
7     external
8 +     revertIfDeadlinePassed(deadline)
9     revertIfZero(wethToDeposit)
10    returns (uint256 liquidityTokensToMint)
11    {
```

Low

[L-1] TSwapPool::_addLiquidityMintAndTransfer emits event with wrong order of Parameters

Description: When the `LiquidityAdded` event is emitted in the `TSwapPool::_addLiquidityMintAndTransfer` function, it logs values in an incorrect order. The `poolTokenToDeposit` value should go in the third parameter's position, whereas the `wethToDeposit` value should go second.

Impact: Event emission is incorrect, leading to off-chain function potentially malfunctioning.

Recommended Mitigation:

```
1 -     emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit)
   ;
2 +     emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit)
   ;
```

[L-2] Default value returned by TSwapPool::swapExactInput result in an incorrect return value given

Description: The `swapExactInput` function is expected to return the actual amount of tokens bought by the caller. However, while it declares the named return value `output` it is never assigned a value, nor does it use an explicit return statement.

Impact: The return value will always be 0, thereby giving an incorrect information (the amount of tokens they get) to the caller.

Recommended Mitigation:

```
1     {
2         uint256 inputReserves = inputToken.balanceOf(address(this))
3         ;
4         uint256 outputReserves = outputToken.balanceOf(address(this));
```

```
5 -         uint256 outputAmount = getOutputAmountBasedOnInput(
6 +         uint256 output = getOutputAmountBasedOnInput(inputAmount,
7         inputReserves, outputReserves);
8 -         if (outputAmount < minOutputAmount) {
9 -             revert TSwapPool__OutputTooLow(outputAmount,
10            minOutputAmount);
11 +         if (output < minOutputAmount) {
12 +             revert TSwapPool__OutputTooLow(outputAmount,
13            minOutputAmount);
14            }
15 -         _swap(inputToken, inputAmount, outputToken, outputAmount);
16 +         _swap(inputToken, input, outputToken, output);
17     }
```

Informationals

[I-1] PoolFactory::PoolFactory__PoolDoesNotExist is not used and should be removed in

```
1 - error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] No zero address check in PoolFactory and TSwapPool constructor

```
1     constructor(address wethToken) {
2 +         if(wethToken == address(0)) {
3 +             revert();
4 +         }
5 -         i_wethToken = wethToken;
6     }
```

```
1     constructor(
2         address poolToken,
3         address wethToken,
4         string memory liquidityTokenName,
5         string memory liquidityTokenSymbol
6     )
7     ERC20(liquidityTokenName, liquidityTokenSymbol)
8     {
9 +         if(poolToken == address(0) && wethToken == address(0)) {
10 +             revert();
11 +         }
12         // written-info zero address check
```

```
13 -     i_wethToken = IERC20(wethToken);  
14 -     i_poolToken = IERC20(poolToken);  
15 }
```

[I-3] PoolFactory::createPool should use .symbol() instead of .name()

Description: When deriving the symbol for a liquidity token, it's best to concat `ts` (representing T-SWAP) with symbol of the underlying pool token (e.g ARB for Arbitrum) and not the full name of the token.

```
1 -     string memory liquidityTokenSymbol = string.concat("ts", IERC20(  
    tokenAddress).name());  
2 +     string memory liquidityTokenSymbol = string.concat("ts", IERC20(  
    tokenAddress).symbol());
```

[I-4] Event is missing indexed fields

Description: Index event fields make the field more quickly accesible to off-chain tools that parse events. Each event should use three indexed fields if there are three or more fields, and gas usage is not particularly of concern for the events in question.

```
1 -     event Swap(address indexed swapper, IERC20 tokenIn, uint256  
    amountTokenIn, IERC20 tokenOut, uint256 amountTokenOut);  
2 +     event Swap(address indexed swapper, IERC20 indexed tokenIn, uint256  
    indexed amountTokenIn, IERC20 tokenOut, uint256 amountTokenOut);
```

[I-5] MINIMUM_WETH_LIQUIDITY is a constant and not required to be emitted

Description: Since `MINIMUM_WETH_LIQUIDITY` is a constant global available with a getter function for evryone to see, it is redundant to emit it alongside other parameters.

```
1 +     revert TSwapPool__WethDepositAmountTooLow(wethToDeposit);  
2 -     revert TSwapPool__WethDepositAmountTooLow(MINIMUM_WETH_LIQUIDITY,  
    wethToDeposit);
```

[I-6] Unused variable in TSwapPool::deposit not gas Efficient

Description: `poolTokenReserves` is never used through out the `deposit` function, hence it is wise to removed it as it is not gas efficient

```
1 -     uint256 poolTokenReserves = i_poolToken.balanceOf(address(this));
```

[I-7] TSwapPool::deposit doesn't follow CEI

Description: This function first adds liquidity and mint LP tokens before updating the `liquidityTokensToMint` variable, thereby leaving window for potential unforeseen re-entrancy attack

```
1 - _addLiquidityMintAndTransfer(wethToDeposit,
2 -   maximumPoolTokensToDeposit, wethToDeposit);
3
4 + liquidityTokensToMint = wethToDeposit;
5 + _addLiquidityMintAndTransfer(wethToDeposit,
   maximumPoolTokensToDeposit, wethToDeposit);
```

[I-8] No natspec in TSwapPool::swapExactInput

Description: Without a clear documentation of what the function does and the all it's parameters, users and developers may mis-interpret the function.

Impact: Without understanding the function, they may Call it with wrong parameters or don't even interact with it at all.

Recommended Mitigation: Provide a detailed natspec for the `swapExactInput` function.

[I-9] Functions not used within the contract should be external

Description: The `swapExactInput` and `swapExactOutput` function is public but it's never used or called within the contract

Recommended Mitigation: Public functions are generally more expensive than externals. Since the function is only called by other addresses or contract when swapping a token it should be marked as external.

```
1     function swapExactInput(
2         IERC20 inputToken,
3         uint256 inputAmount,
4         IERC20 outputToken,
5         uint256 minOutputAmount,
6         uint64 deadline
7     )
8 +     external
9 -     public
10    revertIfZero(inputAmount)
11    revertIfDeadlinePassed(deadline)
12    returns (uint256 output)
```

13	{
----	---